CS5340 Assignment 3:
Hidden Markov Models

1 Overview

In this assignment, you will implement the Baum-Welch algorithm to find the unknown parameters of one-dimensional Gaussian hidden Markov models (HMMs).

Reference: Lecture 8

Honour Code. This coding assignment constitutes **15%** of your final grade in CS5340. Note that plagiarism will not be condoned! You may discuss with your classmates and check the internet for references, but you **must not** submit code or a report copied directly from other sources.

2 Submission Instructions

Submit the following items:

- **Source code (`lab3.py`).** Fill in all your code here.
- **Report (`report.pdf`).** Describe your implementation in no more than one page.

Indicate clearly your name and student number (the one that looks like `A1234567X`) in the report and at the top of your source code. Zip the two files together and name the archive `A1234567X_lab3.zip`, replacing the example with your student number.

Submit your assignment by **18 October 2026** to Canvas. 25% of the total score will be deducted for each day of late submission.

3 Getting Started

This assignment requires Python 3.5 or later. Install the required packages using:

```
pip install -r requirements.txt
```

If you have installation issues, post them in the forum so that other students or the instructors can help accordingly.

4 Test Cases

We provide test cases and expected outputs in the `data/` folder. The code that loads the sample data and checks your output is in `test_lab3.py`. During grading, your code will also be checked

on held-out cases.

For the complete Baum-Welch implementation, outputs can differ slightly across valid implementations. We therefore provide the ground-truth generating parameters and list the parameters obtained by our reference implementation in Appendix A. During grading, the fitted model will be evaluated using its likelihood $p(\mathbf{X} \mid \boldsymbol{\theta})$.

5 Notation

We use the following notation throughout:

- K: number of states.
- N: number of time steps.
- $\boldsymbol{\theta} = \{\boldsymbol{\pi}, \mathbf{A}, \boldsymbol{\phi}\}$: HMM parameters to estimate.
- $\boldsymbol{\pi}$: initial-state distribution, represented by a NumPy array of size K.
- $\mathbf{A}$: time-independent stochastic transition matrix, represented by a $K \times K$ NumPy array.
- $\boldsymbol{\phi} = \{\phi_1, \ldots, \phi_K\}$: emission-distribution parameters. For the Gaussian HMM, these are means $\{\mu_1, \ldots, \mu_K\}$ and standard deviations $\{\sigma_1, \ldots, \sigma_K\}$, each represented by an array of size K.
- $\mathbf{X} = \{\mathbf{x}_1, \ldots, \mathbf{x}_N\}$: observed data.
- $\mathbf{Z} = \{\mathbf{z}_1, \ldots, \mathbf{z}_N\}$: latent states.
- $\gamma(\mathbf{z}_n) = p(\mathbf{z}_n \mid \mathbf{X}, \boldsymbol{\theta}^{\text{old}})$: marginal posterior distribution.
- $\xi(\mathbf{z}_{n-1}, \mathbf{z}_n) = p(\mathbf{z}_{n-1}, \mathbf{z}_n \mid \mathbf{X}, \boldsymbol{\theta}^{\text{old}})$: joint posterior distribution of two successive latent states.

6 Baum-Welch Algorithm

The Baum-Welch algorithm uses expectation-maximization (EM) to find the maximum-likelihood HMM parameters:

$$\operatorname*{argmax}_{\boldsymbol{\theta}} p(\mathbf{X} \mid \boldsymbol{\theta}). \tag{1}$$

Starting from parameters $\boldsymbol{\theta}^{\text{old}}$, EM alternates between an E-step and an M-step. The E-step finds $p(\mathbf{Z} \mid \mathbf{X}, \boldsymbol{\theta}^{\text{old}})$, allowing us to evaluate the expected complete-data log-likelihood

$$\begin{aligned} Q(\boldsymbol{\theta}, \boldsymbol{\theta}^{\text{old}}) &= \mathbb{E}_{\mathbf{Z} \mid \mathbf{X}, \boldsymbol{\theta}^{\text{old}}}[\log p(\mathbf{X}, \mathbf{Z} \mid \boldsymbol{\theta})] \\ &= \sum_{\mathbf{Z}} p(\mathbf{Z} \mid \mathbf{X}, \boldsymbol{\theta}^{\text{old}}) \log p(\mathbf{X}, \mathbf{Z} \mid \boldsymbol{\theta}). \end{aligned} \tag{2}$$

The M-step then maximises Q with respect to $\boldsymbol{\theta}$.

6.1 E-step

For an HMM, the expected complete-data log-likelihood can be written as

$$\begin{aligned} Q(\boldsymbol{\theta}, \boldsymbol{\theta}^{\text{old}}) = & \sum_{k=1}^{K} \gamma(z_{1k}) \log \pi_k \\ & + \sum_{n=2}^{N} \sum_{j=1}^{K} \sum_{k=1}^{K} \xi(z_{n-1,j}, z_{n,k}) \log A_{jk} \\ & + \sum_{n=1}^{N} \sum_{k=1}^{K} \gamma(z_{n,k}) \log p(x_n \mid \phi_k). \end{aligned} \tag{3}$$

Implement `e_step()` using the alpha-beta form of the forward-backward algorithm taught in the lectures. It takes the list of observation sequences $\mathbf{X}$ and current parameters $\boldsymbol{\theta} = \{\boldsymbol{\pi}, \mathbf{A}, \boldsymbol{\phi}\}$, and returns lists of $\gamma(\mathbf{z}_n)$ and $\xi(\mathbf{z}_{n-1}, \mathbf{z}_n)$ values. Compute the outputs separately for each sequence and collect them in lists of the same length as the input list.

Forward probabilities can approach zero rapidly for long sequences. You must implement scaling factors for numerical stability to receive full credit.

Hints:

- Two sequences are provided for checking the E-step. The short sequence works without scaling and can be used before adding scaling factors.
- Use the time indices z_{n-1} and z_n to track which dimensions should be multiplied or summed.

6.2 M-step

Implement `m_step()`, which takes the observations, γ, and ξ and returns the maximum-likelihood estimates $\boldsymbol{\theta} = \{\boldsymbol{\pi}, \mathbf{A}, \boldsymbol{\phi}\}$. This assignment considers only one-dimensional Gaussian HMMs, so $\boldsymbol{\phi}$ contains the means $\{\mu_1, \ldots, \mu_K\}$ and standard deviations $\{\sigma_1, \ldots, \sigma_K\}$.

6.3 Putting the Steps Together

Implement `fit_hmm()` by alternating your E-step and M-step until the parameters converge. Use the provided initialization routine without changing its intended behaviour. It uses a fixed random seed for $\boldsymbol{\pi}$ and $\mathbf{A}$, and k-means clustering of the observations to initialize $\boldsymbol{\phi} = \{\boldsymbol{\mu}, \boldsymbol{\sigma}\}$. The initialized parameters must be used in the first E-step.

Use a reasonable convergence condition. For example, stop when the maximum absolute change across all parameters is less than 10^{-4}. Include a maximum iteration limit so that the procedure also terminates if numerical convergence stalls.

A Training Datasets

The following figures and tables show the models used to generate the supplied observations and the parameters estimated by the reference implementation. State labels may be permuted without changing an HMM's likelihood.

A.1 `seq_short`

This dataset contains 200 sequences, each with 8 time steps.

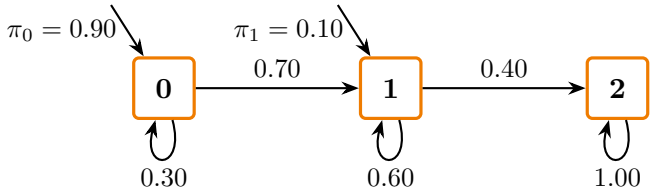


Figure 1: Ground-truth transition structure for `seq_short`.

(a) Ground-truth generating parameters

$\boldsymbol{\pi} = (0.90,\ 0.10,\ 0.00)$

$\mathbf{A}$			Emission parameters			
0.30	0.70	0.00		0	1	2
0.00	0.60	0.40	μ	−1.50	0.50	−0.20
0.00	0.00	1.00	σ	0.50	0.20	0.30

(b) Reference estimates

$\hat{\boldsymbol{\pi}} = (0.91,\ 0.09,\ 0.00)$

$\hat{\mathbf{A}}$			Estimated emission parameters			
0.28	0.72	0.00		0	1	2
0.00	0.60	0.40	$\hat{\mu}$	−1.50	0.52	−0.21
0.00	0.00	1.00	$\hat{\sigma}$	0.51	0.21	0.29

A.2 `seq_long`

This dataset contains 5 sequences, each with 1000 time steps.

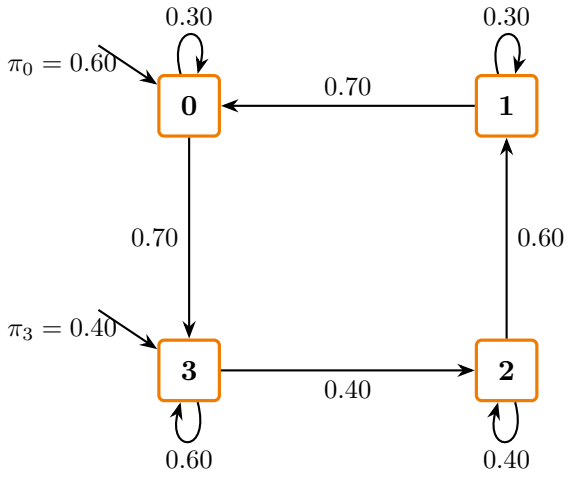


Figure 2: Ground-truth transition structure for `seq_long`.

(a) Ground-truth generating parameters

$\boldsymbol{\pi} = (0.60,\ 0.00,\ 0.00,\ 0.40)$

A			
0.30	0.00	0.00	0.70
0.70	0.30	0.00	0.00
0.00	0.60	0.40	0.00
0.00	0.00	0.40	0.60

Emission parameters

	0	1	2	3
μ	0.00	1.10	2.00	1.00
σ	0.50	0.30	0.40	0.20

(b) Reference estimates

$\hat{\boldsymbol{\pi}} = (0.64,\ 0.00,\ 0.00,\ 0.36)$

$\hat{\mathbf{A}}$			
0.30	0.00	0.00	0.70
0.70	0.30	0.00	0.00
0.00	0.58	0.42	0.00
0.00	0.00	0.43	0.57

Estimated emission parameters

	0	1	2	3
$\hat{\mu}$	0.00	1.07	2.00	1.00
$\hat{\sigma}$	0.52	0.31	0.41	0.19